

Lab0 实验报告

一、思考题

1.编译

使用 x86 原生编译器条件下（以helloworld程序为例） `readelf` 结果如下：

```
ELF 头:
Magic:    7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00
类别:                                ELF64
数据:                                2 补码, 小端序 (little endian)
Version:  1 (current)
OS/ABI:    UNIX - System V
ABI 版本:  0
类型:      REL (可重定位文件)
系统架构:  Advanced Micro Devices X86-64
版本:      0x1
入口点地址:          0x0
程序头起点:          0 (bytes into file)
Start of section headers: 608 (bytes into file)
标志:          0x0
Size of this header:  64 (bytes)
Size of program headers: 0 (bytes)
Number of program headers: 0
Size of section headers: 64 (bytes)
Number of section headers: 14
Section header string table index: 13
```

以 mips-linux-gnu-gcc 交叉编译器编译的 `readelf` 如下

```
ELF 头:
Magic:    7f 45 4c 46 01 02 01 00 00 00 00 00 00 00 00
类别:                                ELF32
数据:                                2 补码, 大端序 (big endian)
Version:  1 (current)
OS/ABI:    UNIX - System V
ABI 版本:  0
类型:      REL (可重定位文件)
系统架构:  MIPS R3000
版本:      0x1
入口点地址:          0x0
程序头起点:          0 (bytes into file)
Start of section headers: 804 (bytes into file)
标志:          0x70001007, noreorder, pic, cpic, o32, mips32r2
Size of this header:  52 (bytes)
```

```

Size of program headers:      0 (bytes)
Number of program headers:    0
Size of section headers:     40 (bytes)
Number of section headers:    17
Section header string table index: 16
string table index: 13

```

通过 `readelf -h` 对比原生 x86 与 MIPS 交叉编译生成的重定位文件 (.o)，主要差异如下：

- **系统架构 (Machine):**
 - **x86-64:** 显示为 **Advanced Micro Devices X86-64**，属复杂指令集 (CISC)。
 - **MIPS:** 显示为 **MIPS R3000** (MOS 实验目标架构)，属精简指令集 (RISC)。
- **字长与类别 (Class):**
 - **x86-64:** **ELF64**，指针和地址长度为 64 位。
 - **MIPS:** **ELF32**，对应 32 位处理器，其 ELF Header (52B) 相比 64 位 (64B) 更小。
- **字节序 (Data):**
 - **x86-64:** 固定的 **Little Endian** (小端序)。
 - **MIPS:** 本次结果显示为 **Big Endian** (大端序)。注：MIPS 架构支持双端序，具体取决于交叉编译器的 **-EL** 或 **-EB** 参数。
- **指令特征 (objdump -d):**
 - **x86:** 指令变长 (1-15 字节)，汇编逻辑复杂。
 - **MIPS:** 指令定长 (4 字节/32-bit)，机器码排列极整齐，便于硬件流水线解码。

objdump 参数含义

在内核开发与调试过程中，各参数功用如下：

- **-d (--disassemble):** * 含义：反汇编代码段 (如 **.text**)。
 - 功用：确认编译器生成的 MIPS 汇编指令是否符合预期。
- **-D (--disassemble-all):** * 含义：反汇编所有节 (包括数据段)。
 - 功用：用于检查数据段、异常向量表等区域是否含有非预期的机器码。
- **-S (--source):** * 含义：将源代码与汇编指令交叉显示。
 - 功用：调试利器。直观观察 C 语言逻辑如何被翻译成具体的寄存器操作。
- **-h (--section-headers):** * 含义：显示节头信息 (包含 VMA 虚拟地址、LMA 加载地址、大小和偏移)。
 - 功用：验证链接脚本 (**kernel.lds**) 是否生效。通过 VMA 确认内核各段是否被安置在 **0x80010000** 等预定地址。
- **-x (--all-headers):** * 含义：显示所有头信息 (包含符号表等)。
 - 功用：查看全局变量、函数符号的具体地址映射。

2. readelf

```

git@24371277:~/24371277/tools/readelf (lab1)$ ./readelf ~/24371277/target/mos
0:0x0
1:0x80400000
2:0x804016f0
3:0x80401708

```

```
4:0x80401720
5:0x0
6:0x0
7:0x0
8:0x0
9:0x0
10:0x0
11:0x0
12:0x0
13:0x0
14:0x0
15:0x0
16:0x0
17:0x0
```

```
git@24371277:~/24371277/tools/readelf (lab1)$ readelf -h
readelf: 警告: 无事可做。
用法: readelf <选项> elf-文件
```

编写的 readelf 能解析 mos (ELF32) , 不能解析自身 (ELF64) 。

- 其原因是实验代码中定义的 Elf32_Ehdr 等结构体只适用于 32 位 ELF 文件。Makefile 使用原生 gcc 编译工具, 导致工具本身是 64 位程序; 而目标内核是交叉编译生成的 32 位 MIPS 程序。
- 系统 readelf 具备架构自适应能力, 会根据 ELF 头中的 EI_CLASS 字段动态调整解析策略。

3. 启动入口

在理论课上我们了解到, MIPS体系结构上电时, 启动入口地址为0xBFC00000

(其实启动入口地址是根据具体型号而定的, 由硬件逻辑确定, 也有可能不是这个地址, 但一定是一个确定的地址)

但实验操作系统的内核入口并没有放在上电启动地址, 而是按照内存布局图放置。思考为什么这样放置内核还能保证内核入口被正确跳转到?

因为实验操作系统 QEMU 已经自带 bootloader 上电启动后已经完成了最初的引导程序, 可以直接使用 mips-linux-gnu-ld 进行链接并配合 kernel.lds 时, 将 _start 符号的虚拟地址写入 ELF 文件的**文件头 (ELF Header) **中, 从而正确跳转

二、难点分析

C语言中全局变量会有默认值0。

- 这是因为操作系统在加载时将所有**未初始化的全局变量所占的内存统一填了0**。

ELF文件的结构

- **ELF 文件头**: 包含程序的基本信息, 以及节头表和段头表相对文件的偏移位置。
- **段头表 (程序头表)**: 包含程序中各个**段**的信息。段的信息需要在运行时装载。

- **节头表**：包含程序中各个节的信息。节的信息需要在**程序编译链接**时使用。
- **段头表（程序头表）的表项**：记录该段数据在文件中的大小、载入内存的位置等，用于指导程序加载。
- **节头表的表项**：记录了该节程序的**文件内偏移**、节地址**等信息，主要是链接器在链接的时候需要使用。

这里需要注意**文件内**是相对于整个ELF文件的 因此，在读取节头信息时

```
int readelf(const void *binary, size_t size) {
    Elf32_Ehdr *ehdr = (Elf32_Ehdr *)binary;
    sh_table = binary + ehdr->e_shoff; //相对于整个文件，即binary的偏移!
}
```

Section (节)

- .text包含了可执行文件中的代码
- .data包含了需要被初始化的全局变量和静态变量
- 而.bss包含了未初始化的全局变量和静态变量。

Linker script

```
SECTIONS
{
    . = 0x10000;
    .text : { *(.text) }
    . = 0x8000000;
    .data : { *(.data) }
    .bss : { *(.bss) }
}
```

∴ **定位计数器 (Location Counter)**，初始值为 0，通过赋值直接指定后续段的起始地址。

- 第 3 行 . = 0x10000;：将后续 .text 段的起始地址设为 0x10000。
- 第 6 行 . = 0x8000000;：将后续 .data 段的起始地址强制设为 0x8000000。

定义内容映射规则

- 例如 .text : { *(.text) }：将所有输入文件的 .text 段合并为输出文件的 .text 段；同理 .data/.bss 段规则一致。
- 注意：**内存表，自下而上生长**，

C语言-变长参数表

示例如下，每次进行va_arg(ap, int)后，ap就会指向下一个存在的参数。

```
#include<stdarg.h>
void myprint(const char *fmt, ...)
{
    va_list ap;
    va_start(ap, lastarg);
    int num;
    num = va_arg(ap, int);
    ...
    va_end(ap);
}
```

三、实验体会

- 基本的库函数搭建时一定要多 debug!! 防止在上机时出现**意想不到的错误**
- 课下搭建可以适当使用 vscode 来加速编辑, 学习。同时也要保持vim的手感, 终端与编辑器同时使用!